

```
!pip install datasets
```

```
Collecting datasets
```

```
  Downloading datasets-3.5.0-py3-none-any.whl.metadata (19 kB)
```

```
Requirement already satisfied: filelock in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (3.18.0)
```

```
Requirement already satisfied: numpy>=1.17 in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (2.0.2)
```

```
Requirement already satisfied: pyarrow>=15.0.0 in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (18.1.0)
```

```
Collecting dill<0.3.9,>=0.3.0 (from datasets)
```

```
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
```

```
Requirement already satisfied: pandas in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (2.2.2)
```

```
Requirement already satisfied: requests>=2.32.2 in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (2.32.3)
```

```
Requirement already satisfied: tqdm>=4.66.3 in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (4.67.1)
```

```
Collecting xxhash (from datasets)
```

```
  Downloading xxhash-3.5.0-cp311-cp311-
```

```
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
```

```
Collecting multiprocess<0.70.17 (from datasets)
```

```
  Downloading multiprocess-0.70.16-py311-none-any.whl.metadata (7.2 kB)
```

```
Collecting fsspec<=2024.12.0,>=2023.1.0 (from
```

```
fsspec[http]<=2024.12.0,>=2023.1.0->datasets)
```

```
  Downloading fsspec-2024.12.0-py3-none-any.whl.metadata (11 kB)
```

```
Requirement already satisfied: aiohttp in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (3.11.15)
```

```
Requirement already satisfied: huggingface-hub>=0.24.0 in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (0.30.2)
```

```
Requirement already satisfied: packaging in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (24.2)
```

```
Requirement already satisfied: pyyaml>=5.1 in
```

```
/usr/local/lib/python3.11/dist-packages (from datasets) (6.0.2)
```

```
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in
```

```
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)  
(2.6.1)
```

```
Requirement already satisfied: aiosignal>=1.1.2 in
```

```
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)  
(1.3.2)
```

```
Requirement already satisfied: attrs>=17.3.0 in
```

```
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)  
(25.3.0)
```

```
Requirement already satisfied: frozenlist>=1.1.1 in
```

```
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)  
(1.5.0)
```

```
Requirement already satisfied: multidict<7.0,>=4.5 in
```

```
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)  
(6.4.2)
```

```

Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)
(0.3.1)
Requirement already satisfied: yarll<2.0,>=1.17.0 in
/usr/local/lib/python3.11/dist-packages (from aiohttp->datasets)
(1.19.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in
/usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.24.0-
>datasets) (4.13.1)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.32.2-
>datasets) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.32.2-
>datasets) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.32.2-
>datasets) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests>=2.32.2-
>datasets) (2025.1.31)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/dist-packages (from pandas->datasets)
(2.8.2)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.11/dist-packages (from pandas->datasets)
(2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.11/dist-packages (from pandas->datasets)
(2025.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets) (1.17.0)
Downloading datasets-3.5.0-py3-none-any.whl (491 kB)
----- 491.2/491.2 kB 10.9 MB/s eta
0:00:00
----- 116.3/116.3 kB 8.6 MB/s eta
0:00:00
----- 183.9/183.9 kB 13.7 MB/s eta
0:00:00
ultiprocess-0.70.16-py311-none-any.whl (143 kB)
----- 143.5/143.5 kB 10.5 MB/s eta
0:00:00
anylinux_2_17_x86_64.manylinux2014_x86_64.whl (194 kB)
----- 194.8/194.8 kB 13.5 MB/s eta
0:00:00
ultiprocess, datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2025.3.2

```

Uninstalling fsspec-2025.3.2:

Successfully uninstalled fsspec-2025.3.2

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

torch 2.6.0+cu124 requires nvidia-cublas-cu12==12.4.5.8; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cublas-cu12 12.5.3.2 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cuda-cupti-cu12==12.4.127; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cuda-cupti-cu12 12.5.82 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cuda-nvrtc-cu12==12.4.127; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cuda-nvrtc-cu12 12.5.82 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cuda-runtime-cu12==12.4.127; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cuda-runtime-cu12 12.5.82 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cudnn-cu12==9.1.0.70; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cudnn-cu12 9.3.0.75 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cufft-cu12==11.2.1.3; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cufft-cu12 11.2.3.61 which is incompatible.

torch 2.6.0+cu124 requires nvidia-curand-cu12==10.3.5.147; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-curand-cu12 10.3.6.82 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cusolver-cu12==11.6.1.9; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cusolver-cu12 11.6.3.83 which is incompatible.

torch 2.6.0+cu124 requires nvidia-cuspars-cu12==12.3.1.170; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-cuspars-cu12 12.5.1.3 which is incompatible.

torch 2.6.0+cu124 requires nvidia-nvjitlink-cu12==12.4.127; platform_system == "Linux" and platform_machine == "x86_64", but you have nvidia-nvjitlink-cu12 12.5.82 which is incompatible.

gcsfs 2025.3.2 requires fsspec==2025.3.2, but you have fsspec 2024.12.0 which is incompatible.

Successfully installed datasets-3.5.0 dill-0.3.8 fsspec-2024.12.0 multiprocessing-0.70.16 xxhash-3.5.0

!pip install bertviz

Collecting bertviz

Downloading bertviz-1.4.0-py3-none-any.whl.metadata (19 kB)

Requirement already satisfied: transformers>=2.0 in /usr/local/lib/python3.11/dist-packages (from bertviz) (4.51.1)

Requirement already satisfied: torch>=1.0 in /usr/local/lib/python3.11/dist-packages (from bertviz) (2.6.0+cu124)

Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from bertviz) (4.67.1)

```
Collecting boto3 (from bertviz)
  Downloading boto3-1.37.36-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: requests in
/usr/local/lib/python3.11/dist-packages (from bertviz) (2.32.3)
Requirement already satisfied: regex in
/usr/local/lib/python3.11/dist-packages (from bertviz) (2024.11.6)
Requirement already satisfied: sentencepiece in
/usr/local/lib/python3.11/dist-packages (from bertviz) (0.2.0)
Requirement already satisfied: filelock in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)
(3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)
(4.13.1)
Requirement already satisfied: networkx in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)
(3.4.2)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)
(3.1.6)
Requirement already satisfied: fsspec in
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)
(2024.12.0)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch>=1.0->bertviz)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch>=1.0-
>bertviz)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch>=1.0->bertviz)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch>=1.0->bertviz)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch>=1.0->bertviz)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch>=1.0->bertviz)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.5.147 (from torch>=1.0->bertviz)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch>=1.0->bertviz)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cuspars-cu12==12.3.1.170 (from torch>=1.0->bertviz)
```

```
Downloading nvidia_cusparselt-cu12-12.3.1.170-py3-none-  
manylinux2014_x86_64.whl.metadata (1.6 kB)  
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in  
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)  
(0.6.2)  
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in  
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)  
(2.21.5)  
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in  
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)  
(12.4.127)  
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch>=1.0->bertviz)  
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-  
manylinux2014_x86_64.whl.metadata (1.5 kB)  
Requirement already satisfied: triton==3.2.0 in  
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)  
(3.2.0)  
Requirement already satisfied: sympy==1.13.1 in  
/usr/local/lib/python3.11/dist-packages (from torch>=1.0->bertviz)  
(1.13.1)  
Requirement already satisfied: mpmath<1.4,>=1.1.0 in  
/usr/local/lib/python3.11/dist-packages (from sympy==1.13.1-  
>torch>=1.0->bertviz) (1.3.0)  
Requirement already satisfied: huggingface-hub<1.0,>=0.30.0 in  
/usr/local/lib/python3.11/dist-packages (from transformers>=2.0-  
>bertviz) (0.30.2)  
Requirement already satisfied: numpy>=1.17 in  
/usr/local/lib/python3.11/dist-packages (from transformers>=2.0-  
>bertviz) (2.0.2)  
Requirement already satisfied: packaging>=20.0 in  
/usr/local/lib/python3.11/dist-packages (from transformers>=2.0-  
>bertviz) (24.2)  
Requirement already satisfied: pyyaml>=5.1 in  
/usr/local/lib/python3.11/dist-packages (from transformers>=2.0-  
>bertviz) (6.0.2)  
Requirement already satisfied: tokenizers<0.22,>=0.21 in  
/usr/local/lib/python3.11/dist-packages (from transformers>=2.0-  
>bertviz) (0.21.1)  
Requirement already satisfied: safetensors>=0.4.3 in  
/usr/local/lib/python3.11/dist-packages (from transformers>=2.0-  
>bertviz) (0.5.3)  
Collecting botocore<1.38.0,>=1.37.36 (from boto3->bertviz)  
  Downloading botocore-1.37.36-py3-none-any.whl.metadata (5.7 kB)  
Collecting jmespath<2.0.0,>=0.7.1 (from boto3->bertviz)  
  Downloading jmespath-1.0.1-py3-none-any.whl.metadata (7.6 kB)  
Collecting s3transfer<0.12.0,>=0.11.0 (from boto3->bertviz)  
  Downloading s3transfer-0.11.5-py3-none-any.whl.metadata (1.7 kB)  
Requirement already satisfied: charset-normalizer<4,>=2 in  
/usr/local/lib/python3.11/dist-packages (from requests->bertviz)
```

```

(3.4.1)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.11/dist-packages (from requests->bertviz)
(3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests->bertviz)
(2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests->bertviz)
(2025.1.31)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in
/usr/local/lib/python3.11/dist-packages (from
botocore<1.38.0,>=1.37.36->boto3->bertviz) (2.8.2)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.11/dist-packages (from jinja2->torch>=1.0-
>bertviz) (3.0.2)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.11/dist-packages (from python-
dateutil<3.0.0,>=2.1->botocore<1.38.0,>=1.37.36->boto3->bertviz)
(1.17.0)
Downloading bertviz-1.4.0-py3-none-any.whl (157 kB)
_____ 157.6/157.6 kB 7.1 MB/s eta
0:00:00
anylinux2014_x86_64.whl (363.4 MB)
_____ 363.4/363.4 MB 3.7 MB/s eta
0:00:00
anylinux2014_x86_64.whl (13.8 MB)
_____ 13.8/13.8 MB 52.5 MB/s eta
0:00:00
anylinux2014_x86_64.whl (24.6 MB)
_____ 24.6/24.6 MB 51.4 MB/s eta
0:00:00
e_cul2-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
_____ 883.7/883.7 kB 29.1 MB/s eta
0:00:00
anylinux2014_x86_64.whl (664.8 MB)
_____ 664.8/664.8 MB 2.9 MB/s eta
0:00:00
anylinux2014_x86_64.whl (211.5 MB)
_____ 211.5/211.5 MB 5.8 MB/s eta
0:00:00
anylinux2014_x86_64.whl (56.3 MB)
_____ 56.3/56.3 MB 7.5 MB/s eta
0:00:00
anylinux2014_x86_64.whl (127.9 MB)
_____ 127.9/127.9 MB 7.6 MB/s eta
0:00:00
anylinux2014_x86_64.whl (207.5 MB)
_____ 207.5/207.5 MB 6.1 MB/s eta

```

```
0:00:00
anylinux2014_x86_64.whl (21.1 MB)
21.1/21.1 MB 24.2 MB/s eta
0:00:00
139.9/139.9 kB 8.2 MB/s eta
0:00:00
13.5/13.5 MB 26.0 MB/s eta
0:00:00
espath-1.0.1-py3-none-any.whl (20 kB)
Downloading s3transfer-0.11.5-py3-none-any.whl (84 kB)
84.8/84.8 kB 6.2 MB/s eta
0:00:00
e-cul2, nvidia-cuda-nvrtc-cul2, nvidia-cuda-cupti-cul2, nvidia-cublas-
cul2, jmespath, nvidia-cuspars-cul2, nvidia-cudnn-cul2, botocore,
s3transfer, nvidia-cusolver-cul2, boto3, bertviz
Attempting uninstall: nvidia-nvjitlink-cul2
Found existing installation: nvidia-nvjitlink-cul2 12.5.82
Uninstalling nvidia-nvjitlink-cul2-12.5.82:
Successfully uninstalled nvidia-nvjitlink-cul2-12.5.82
Attempting uninstall: nvidia-curand-cul2
Found existing installation: nvidia-curand-cul2 10.3.6.82
Uninstalling nvidia-curand-cul2-10.3.6.82:
Successfully uninstalled nvidia-curand-cul2-10.3.6.82
Attempting uninstall: nvidia-cufft-cul2
Found existing installation: nvidia-cufft-cul2 11.2.3.61
Uninstalling nvidia-cufft-cul2-11.2.3.61:
Successfully uninstalled nvidia-cufft-cul2-11.2.3.61
Attempting uninstall: nvidia-cuda-runtime-cul2
Found existing installation: nvidia-cuda-runtime-cul2 12.5.82
Uninstalling nvidia-cuda-runtime-cul2-12.5.82:
Successfully uninstalled nvidia-cuda-runtime-cul2-12.5.82
Attempting uninstall: nvidia-cuda-nvrtc-cul2
Found existing installation: nvidia-cuda-nvrtc-cul2 12.5.82
Uninstalling nvidia-cuda-nvrtc-cul2-12.5.82:
Successfully uninstalled nvidia-cuda-nvrtc-cul2-12.5.82
Attempting uninstall: nvidia-cuda-cupti-cul2
Found existing installation: nvidia-cuda-cupti-cul2 12.5.82
Uninstalling nvidia-cuda-cupti-cul2-12.5.82:
Successfully uninstalled nvidia-cuda-cupti-cul2-12.5.82
Attempting uninstall: nvidia-cublas-cul2
Found existing installation: nvidia-cublas-cul2 12.5.3.2
Uninstalling nvidia-cublas-cul2-12.5.3.2:
Successfully uninstalled nvidia-cublas-cul2-12.5.3.2
Attempting uninstall: nvidia-cuspars-cul2
Found existing installation: nvidia-cuspars-cul2 12.5.1.3
Uninstalling nvidia-cuspars-cul2-12.5.1.3:
Successfully uninstalled nvidia-cuspars-cul2-12.5.1.3
Attempting uninstall: nvidia-cudnn-cul2
Found existing installation: nvidia-cudnn-cul2 9.3.0.75
```

```
Uninstalling nvidia-cudnn-cu12-9.3.0.75:  
Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
```

Task 1: Load Dataset

```
import numpy as np  
from datasets import load_dataset  
train_datasets = load_dataset('ag_news', split='train')  
test_dataset = load_dataset('ag_news', split='test')  
  
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/  
_auth.py:94: UserWarning:  
The secret `HF_TOKEN` does not exist in your Colab secrets.  
To authenticate with the Hugging Face Hub, create a token in your  
settings tab (https://huggingface.co/settings/tokens), set it as  
secret in your Google Colab and restart your session.  
You will be able to reuse this secret in all of your notebooks.  
Please note that authentication is recommended but still optional to  
access public models or datasets.  
    warnings.warn(  
  
{  
  "model_id": "59359d0db80d40a284c74596b7dfe2eb",  
  "version_major": 2,  
  "version_minor": 0  
}  
  
{  
  "model_id": "1cbea60884264970ab9adedfa7ea0980",  
  "version_major": 2,  
  "version_minor": 0  
}  
  
{  
  "model_id": "894af9b57e5345cc8ae702283cd8f98c",  
  "version_major": 2,  
  "version_minor": 0  
}  
  
{  
  "model_id": "2ec9a173620c48eea1fa9c6002361397",  
  "version_major": 2,  
  "version_minor": 0  
}  
  
{  
  "model_id": "fae3e95e33dc49f9adb90bde21195931",  
  "version_major": 2,  
  "version_minor": 0  
}  
  
train_valid_split = train_datasets.train_test_split(test_size=0.2,  
seed=42)  
# setting seed for reproducibility  
train_data = train_valid_split['train'] # (96000 samples)  
valid_data = train_valid_split['test'] # validation set (24000  
samples)  
  
import seaborn as sns  
import pandas as pd  
  
def extract_labels(data, split_name):  
    return pd.DataFrame({  
        'label': data['label'],  
        'split': split_name  
    })
```



```

train_df = extract_labels(train_data, 'train')
valid_df = extract_labels(valid_data, 'validation')
test_df = extract_labels(test_dataset, 'test')

import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)

# Plot for each dataset split
sns.countplot(data=train_df, x='label', ax=axes[0], palette='pastel')
axes[0].set_title('Training Set')
axes[0].set_xlabel('')

sns.countplot(data=valid_df, x='label', ax=axes[1], palette='pastel')
axes[1].set_title('Validation Set')
axes[1].set_xlabel('')

sns.countplot(data=test_df, x='label', ax=axes[2], palette='pastel')
axes[2].set_title('Test Set')
axes[2].set_xlabel('')

# Common formatting
for ax in axes:
    ax.set_ylabel('Count')
    ax.set_xticklabels(ax.get_xticklabels(), rotation=45)

plt.suptitle("Class Distribution Across Dataset Splits", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

```

<ipython-input-8-8c8bc6cf2333>:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.countplot(data=train_df, x='label', ax=axes[0],
palette='pastel')

```

<ipython-input-8-8c8bc6cf2333>:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.countplot(data=valid_df, x='label', ax=axes[1],
palette='pastel')

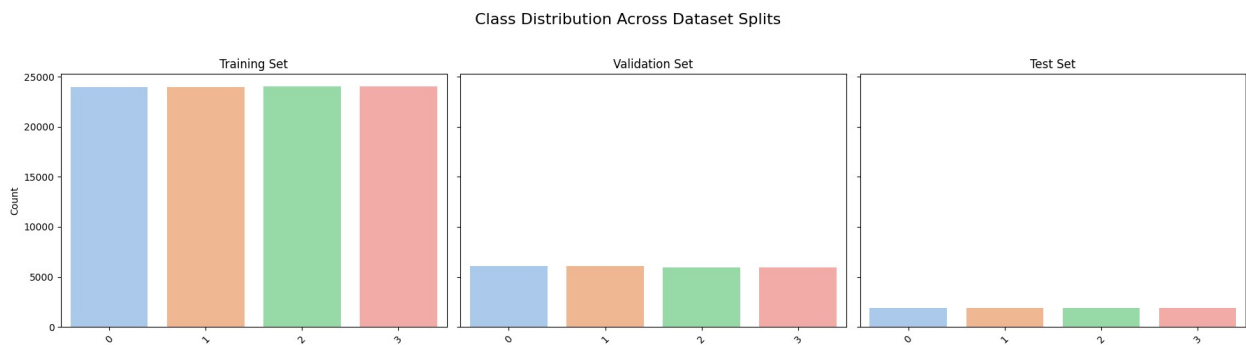
```

<ipython-input-8-8c8bc6cf2333>:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be

removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=test_df, x='label', ax=axes[2], palette='pastel')
<ipython-input-8-8c8bc6cf2333>:21: UserWarning: set_ticklabels()
should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
ax.set_xticklabels(ax.get_xticklabels(), rotation=45)
<ipython-input-8-8c8bc6cf2333>:21: UserWarning: set_ticklabels()
should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
ax.set_xticklabels(ax.get_xticklabels(), rotation=45)
<ipython-input-8-8c8bc6cf2333>:21: UserWarning: set_ticklabels()
should only be used with a fixed number of ticks, i.e. after
set_ticks() or using a FixedLocator.
ax.set_xticklabels(ax.get_xticklabels(), rotation=45)
```



Task 2: Load pre-trained BERT model

```
from transformers import BertTokenizer, BertModel
# Load tokenizer and model
tokenizer = BertTokenizer.from_pretrained("google-bert/bert-base-uncased")
model = BertModel.from_pretrained("google-bert/bert-base-uncased")

{"model_id": "d28eb8156ccd48068d939143e09f277f", "version_major": 2, "version_minor": 0}

{"model_id": "109fd3553ca44e9cb6d3793464f44d19", "version_major": 2, "version_minor": 0}

{"model_id": "3f48ae3bb8db4b21aad6b753ee34df0f", "version_major": 2, "version_minor": 0}

{"model_id": "2e32ab38f4be4567bef8c616f4abde13", "version_major": 2, "version_minor": 0}
```

Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better

performance, install the package with: ``pip install huggingface_hub[hf_xet]`` or ``pip install hf_xet``
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, install the package with: ``pip install huggingface_hub[hf_xet]`` or ``pip install hf_xet``

```
{"model_id":"59664ac293b14c3693b8b6ab06a00e60","version_major":2,"version_minor":0}
```

Task 3: Run experiments

3.1 Probing

Freeze BERT model

```
for param in model.parameters():
    param.requires_grad = False

import torch
from tqdm import tqdm
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, GridSearchCV
import pandas as pd

def get_bert_embedding(text, strategy='cls'):
    inputs = tokenizer(text, return_tensors="pt", padding=True,
truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model(**inputs)

    last_hidden_state = outputs.last_hidden_state # (batch_size,
seq_len, hidden_size)
    attention_mask = inputs['attention_mask']

    if strategy == 'cls':
        # [CLS] token (first token)
        return last_hidden_state[:, 0, :].squeeze(0) # shape:
(hidden_size,)

    elif strategy == 'mean':
        # mean pooling (excluding padding)
        mask = attention_mask.unsqueeze(-
1).expand(last_hidden_state.size()).float()
        sum_embeddings = torch.sum(last_hidden_state * mask, dim=1)
        sum_mask = torch.clamp(mask.sum(1), min=1e-9)
        return (sum_embeddings / sum_mask).squeeze(0) # shape:
(hidden_size,)
```

```

    elif strategy == 'first':
        return last_hidden_state[:, 1, :].squeeze(0) # [CLS] is 0, so
1 is first actual token

    elif strategy == 'last':
        # get last non-padded token
        lengths = attention_mask.sum(dim=1) - 1 # minus 1 for zero-
indexing
        return last_hidden_state[0, lengths, :] # shape:
(hidden_size,)

    else:
        raise ValueError("Invalid strategy selected.")

def extract_embeddings(dataset, strategy='cls', num_samples=1000): #
limit to 1000 for speed
    embeddings = []
    labels = []
    for sample in tqdm(dataset.select(range(num_samples))):
        text = sample['text']
        label = sample['label']
        embedding = get_bert_embedding(text, strategy)
        embeddings.append(embedding.numpy())
        labels.append(label)
    return np.array(embeddings), np.array(labels)

```

Now that our extraction functions are defined, let's use the validation set to determine which strategy is the best between average, cls, first, and last

```

X_train_mean, y_train_mean = extract_embeddings(valid_data,
strategy='mean', num_samples=5000)
X_test_mean, y_test_mean = extract_embeddings(valid_data,
strategy='mean', num_samples=1000)

100%|██████████| 5000/5000 [14:59<00:00, 5.56it/s]
100%|██████████| 1000/1000 [02:47<00:00, 5.99it/s]

X_train_cls, y_train_cls = extract_embeddings(valid_data,
strategy='cls', num_samples=5000)
X_test_cls, y_test_cls = extract_embeddings(valid_data,
strategy='cls', num_samples=1000)

100%|██████████| 5000/5000 [14:09<00:00, 5.89it/s]
100%|██████████| 1000/1000 [02:48<00:00, 5.94it/s]

X_train_first, y_train_first = extract_embeddings(valid_data,
strategy='first', num_samples=5000)
X_test_first, y_test_first = extract_embeddings(valid_data,
strategy='first', num_samples=1000)

```

```
100%|██████████| 5000/5000 [16:49<00:00, 4.95it/s]
100%|██████████| 1000/1000 [03:00<00:00, 5.56it/s]
```

```
X_train_last, y_train_last = extract_embeddings(valid_data,
strategy='last', num_samples=5000)
X_test_last, y_test_last = extract_embeddings(valid_data,
strategy='last', num_samples=1000)
```

```
100%|██████████| 5000/5000 [14:53<00:00, 5.60it/s]
100%|██████████| 1000/1000 [02:55<00:00, 5.69it/s]
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, GridSearchCV
```

Use validation set to find best K

```
strategy_accuracies_knn = {'strategy': [], 'k': [], 'accuracy': []}

strategies = ['mean', 'cls', 'first', 'last']
strategy_datasets = {
    'mean': (X_train_mean, y_train_mean, X_test_mean, y_test_mean),
    'cls': (X_train_cls, y_train_cls, X_test_cls, y_test_cls),
    'first': (X_train_first, y_train_first, X_test_first,
y_test_first), 'last': (X_train_last, y_train_last, X_test_last,
y_test_last)
}
# put the appropriate datasets into a dictionary for reference later
k_options = range(3,11)

def evaluate_acc(preds, labels):
    return np.mean(preds == labels)

# we iterate through each combination of k and strategy and see which
has the best performance on that strategy's respective dataset
extracted from the validation set
for strat in strategies:
    X_train, y_train, X_test, y_test = strategy_datasets[strat]
    for k in k_options:
        knn = KNeighborsClassifier(n_neighbors=k)
        X_train = X_train.reshape(X_train.shape[0], -1)
        X_test = X_test.reshape(X_test.shape[0], -1)
        knn.fit(X_train, y_train)
        preds = knn.predict(X_test)
        acc = evaluate_acc(preds, y_test)
        strategy_accuracies_knn['strategy'].append(strat)
        strategy_accuracies_knn['k'].append(k)
        strategy_accuracies_knn['accuracy'].append(acc)
```

```

knn_strat_df = pd.DataFrame(strategy_accuracies_knn)
display(knn_strat_df)
knn_strat_df.to_csv('knn_strategy_results.csv')

{"summary":{"\n  \"name\": \"knn_strat_df\", \"rows\": 32,\n  \"fields\": [\n    {\n      \"column\": \"strategy\", \n      \"properties\": {\n        \"dtype\": \"category\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"cls\", \n          \"last\", \n          \"mean\", \n          \"description\": \"\"\"\" \n        ], \n        \"semantic_type\": \"\", \n        \"column\": \"k\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 2, \n          \"min\": 3, \n          \"max\": 10, \n          \"num_unique_values\": 8, \n          \"samples\": [\n            4, \n            8, \n            3 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\"\" \n        }, \n        \"column\": \"accuracy\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": \n0.04791709910978308, \n          \"min\": 0.768, \n          \"max\": \n0.934, \n          \"num_unique_values\": 32, \n          \"samples\": [\n            0.86, \n            0.877, \n            0.896 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\"\"\" \n        } \n      ] \n    } \n  ], \"type\": \"dataframe\", \"variable_name\": \"knn_strat_df\"}

```

It seems like the combination of $k=3$ with a strategy of 'mean' generates the best accuracy according to vectors generated from the validation set

```

strategy_accuracies_lr = {'strategy': [], 'accuracy': []}

for strat in strategies:
    X_train, y_train, X_test, y_test = strategy_datasets[strat]
    X_train = X_train.reshape(X_train.shape[0], -1)
    X_test = X_test.reshape(X_test.shape[0], -1)
    model = LogisticRegression(penalty='l2', max_iter=1000)
    preds = model.fit(X_train, y_train).predict(X_test)
    acc = evaluate_acc(preds, y_test)
    strategy_accuracies_lr['strategy'].append(strat)
    strategy_accuracies_lr['accuracy'].append(acc)

lr_strat_df = pd.DataFrame(strategy_accuracies_lr)
display(lr_strat_df)

{"summary":{"\n  \"name\": \"lr_strat_df\", \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"strategy\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"cls\", \n          \"last\", \n          \"mean\", \n          \"description\": \"\"\"\" \n        ], \n        \"semantic_type\": \"\", \n        \"column\": \"accuracy\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": \n0.015326991442115022, \n          \"min\": 0.94, \n          \"max\": \n

```

```
0.977,\n          \"num_unique_values\": 4,\n          \"samples\": [\n0.977,\n          0.94,\n          0.964\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\"\n          }\n          }\n          ]\n          }\", \"type\": \"dataframe\", \"variable_name\": \"lr_strat_df\"}
```

3.2 Fine-tuning BERT

```
dataset = load_dataset("ag_news")
subset = dataset["train"].shuffle(seed=42).select(range(12000)) # 10k
train, 2k val
train_val_split = subset.train_test_split(test_size=2000, seed=42)
train_ds = train_val_split['train']
val_ds = train_val_split['test']
# we hold out a validation set to monitor the validation loss when
fine-tuning -- we stop if it the validation loss is less than a
certain threshold

tokenizer2 = BertTokenizer.from_pretrained("bert-base-uncased")

def tokenize(sample):
    return tokenizer2(sample['text'], truncation=True,
padding='max_length', max_length=128)
    # use max_length of 128 since we suspect that news headlines
shouldn't be overly long

# function to tokenize a document/sample

train_ds = train_ds.map(tokenize, batched=True)
val_ds = val_ds.map(tokenize, batched=True)
# mapping/transforming training and validation set via tokenizer in
batches (SGD)

train_ds.set_format(type='torch', columns=['text', 'input_ids',
'attention_mask', 'label'])
val_ds.set_format(type='torch', columns=['text', 'input_ids',
'attention_mask', 'label'])

{"model_id": "940683525a18463da6707b742bf874b3", "version_major": 2, "version_minor": 0}

{"model_id": "251f46721e894a25882b0aee4bf38761", "version_major": 2, "version_minor": 0}

from transformers import BertForSequenceClassification

model2 = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)

Some weights of BertForSequenceClassification were not initialized
from the model checkpoint at bert-base-uncased and are newly
initialized: ['classifier.bias', 'classifier.weight']
```

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score
import numpy as np

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, preds)}

training_args = TrainingArguments(
    output_dir="./bert-agnews-ft",
    evaluation_strategy="epoch",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
    save_strategy="epoch",
    load_best_model_at_end=True,
    report_to="none",
)

/usr/local/lib/python3.11/dist-packages/transformers/
training_args.py:1611: FutureWarning: `evaluation_strategy` is
deprecated and will be removed in version 4.46 of transformers. Use
`eval_strategy` instead
    warnings.warn(

trainer = Trainer(
    model=model2,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    tokenizer=tokenizer2,
    compute_metrics=compute_metrics
)

trainer.train()

<ipython-input-27-5b2d4f5b30b8>:1: FutureWarning: `tokenizer` is
deprecated and will be removed in version 5.0.0 for
`Trainer.__init__`. Use `processing_class` instead.
    trainer = Trainer(

<IPython.core.display.HTML object>
```



```
TrainOutput(global_step=1250, training_loss=0.27575840454101563,
metrics={'train_runtime': 502.3522, 'train_samples_per_second':
39.813, 'train_steps_per_second': 2.488, 'total_flos':
1315578900480000.0, 'train_loss': 0.27575840454101563, 'epoch': 2.0})
```

Now that we've finished training the model, we now evaluate its performance using the test set

```
test_dataset = load_dataset("ag_news", split="test")

test_ds = test_dataset.map(tokenize, batched=True)
test_ds.set_format(type='torch', columns=['text', 'input_ids',
'attention_mask', 'label'])

{"model_id": "de94ae8a18c04438b09f6f28a40ab4fd", "version_major": 2, "version_minor": 0}

metrics = trainer.evaluate(eval_dataset=test_ds)
print(metrics)

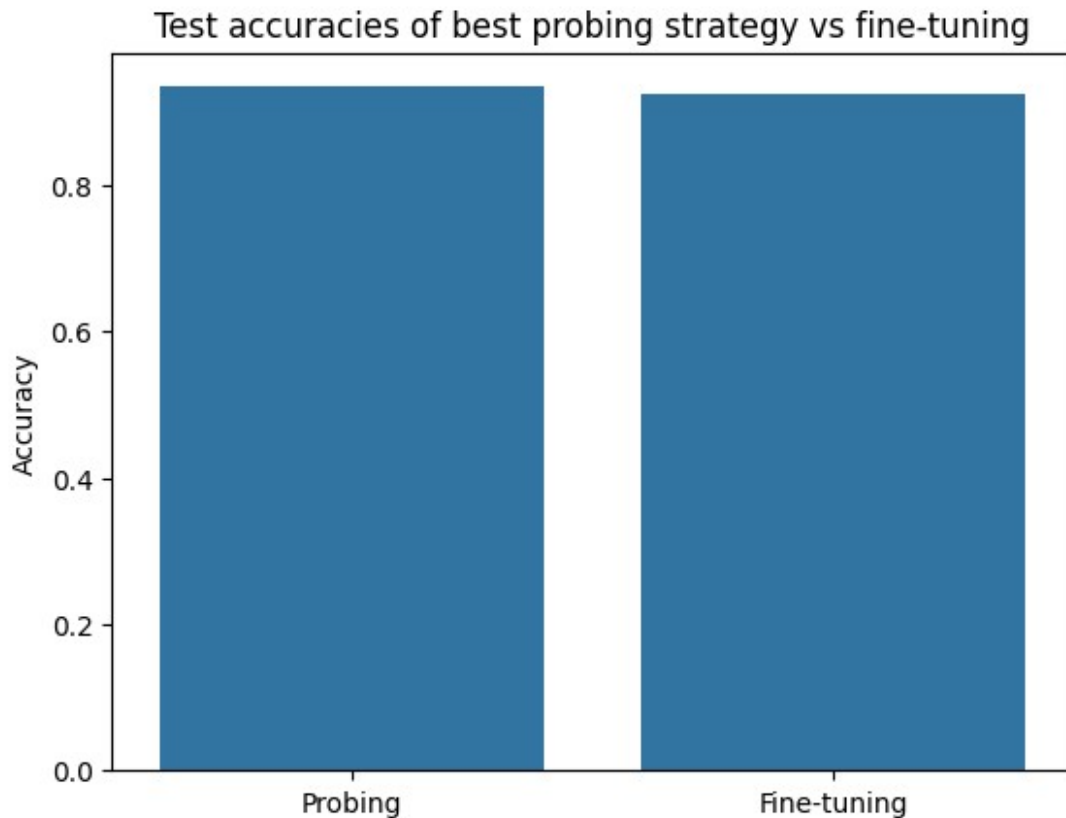
<IPython.core.display.HTML object>

{'eval_loss': 0.26323646306991577, 'eval_accuracy':
0.9232894736842105, 'eval_runtime': 53.3991,
'eval_samples_per_second': 142.324, 'eval_steps_per_second': 8.895,
'epoch': 2.0}
```

The model achieves an accuracy of around 92.3%, not as performant as probing, but not a far cry either, it is still very solid in terms of accuracy

```
# barplot displaying accuracies of probing vs fine-tuning
sns.barplot(x=['Probing', 'Fine-tuning'], y=[0.934,
0.9232894736842105])
plt.title('Test accuracies of best probing strategy vs fine-tuning')
plt.ylabel('Accuracy')

Text(0, 0.5, 'Accuracy')
```



3.4 Attention matrix visualization

Since there are 4 classes, and we need an example of an attention matrix for a positive and negative example for each class, we will need a total of 8 attention matrices. To this end, let's use the validation set to generate predictions and get a positive and negative example for each class

```
import torch
import numpy as np
from torch.utils.data import DataLoader
from sklearn.metrics import accuracy_score

# define device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model2.to(device)

# create DataLoader
val_loader = DataLoader(val_ds, batch_size=16)

# collect predictions and labels with the raw texts
def get_predictions_with_text(model, dataloader, dataset, device):
    model.eval()
    all_preds = []
    all_labels = []
```

```

all_texts = []

for i, batch in enumerate(dataloader):
    input_ids = batch['input_ids'].to(device)
    attention_mask = batch['attention_mask'].to(device)
    labels = batch['label'].to(device)

    with torch.no_grad():
        outputs = model(input_ids=input_ids,
attention_mask=attention_mask)
        logits = outputs.logits
        preds = torch.argmax(logits, dim=-1)

    all_preds.extend(preds.cpu().tolist())
    all_labels.extend(labels.cpu().tolist())

    # get the original texts (match batch idx to dataset)
    start_idx = i * dataloader.batch_size
    end_idx = start_idx + len(labels)
    texts = dataset[start_idx:end_idx]['text']
    all_texts.extend(texts)

return all_preds, all_labels, all_texts

# run prediction
preds, labels, texts = get_predictions_with_text(model2, val_loader,
val_ds, device)

# extract one correct and one incorrect example per class
examples_by_class = {i: {'correct': None, 'incorrect': None} for i in
range(4)}

for text, pred, true_label in zip(texts, preds, labels):
    if pred == true_label and examples_by_class[true_label]['correct']
is None:
        examples_by_class[true_label]['correct'] = (text, pred,
true_label)
    elif pred != true_label and examples_by_class[true_label]
['incorrect'] is None:
        examples_by_class[true_label]['incorrect'] = (text, pred,
true_label)

    # stop early if we found all
    if all(v['correct'] and v['incorrect'] for v in
examples_by_class.values()):
        break

# print results
for cls, example in examples_by_class.items():
    print(f"\nClass {cls}:")

```

```
print(f"Correct:\n Text: {example['correct'][0]}\n Predicted: {example['correct'][1]}\n True: {example['correct'][2]}")
print(f"Incorrect:\n Text: {example['incorrect'][0]}\n Predicted: {example['incorrect'][1]}\n True: {example['incorrect'][2]}")
```

Class 0:

Correct:

Text: Israel Hints Ousting Arafat Delayed by Gaza Pullout JERUSALEM (Reuters) - Israel renewed its threat on Monday to remove Yasser Arafat but hinted it had delayed taking action against the Palestinian president to avoid complicating a planned withdrawal from the Gaza Strip.

Predicted: 0

True: 0

Incorrect:

Text: France Telecom unions call for strike to protest privatization ... PARIS : French trade unions called on workers at France Telecom to stage a 24-hour strike September 7 to protest government plans to privatize the public telecommunications operator, union sources said.

Predicted: 2

True: 0

Class 1:

Correct:

Text: Woodgate out to prove his worth at Real Madrid Jonathan Woodgate says he has learned from his past mistakes off the pitch and is now determined to make the headlines solely for his footballing prowess after completing a dream move to Real Madrid.

Predicted: 1

True: 1

Incorrect:

Text: Back in the swim: Peirsol, Coughlin give Americans much needed splash ATHENS - As the final leg of the men's 4 x 200 freestyle relay began last night at the Olympic pool, the United States held a lead of at least 1 1/2 body lengths - which, considering the final 200 meters was being swum by 6-foot-6 Klete ...

Predicted: 0

True: 1

Class 2:

Correct:

Text: Oil Prices Fall Nearly \$4 Over Past Week Oil prices bounced higher Friday following two days of sharp declines, but traders said they expect prices to move lower again by early next week in anticipation of the Energy Department's next petroleum supply report.

Predicted: 2

True: 2

❑ Incorrect:

Text: Microsoft, Cisco Shake on Network Security In a deal sure to bring smiles to the faces of enterprise security pros, Microsoft (Quote, Chart) and Cisco Systems plan to integrate technologies and push for an industry standard to power network security and health policy compliance.

Predicted: 3

True: 2

Class 3:

❑ Correct:

Text: Earthquake Hits Mount St. Helens Scientists from the US Geological Survey report that small earthquakes have registered about once a minute for the past several weeks.

Predicted: 3

True: 3

❑ Incorrect:

Text: General Mills to Make All Cereals Whole Grain By JOSHUA FREED MINNEAPOLIS (AP) -- The Trix Rabbit and that Lucky Charms leprechaun are going on a whole-grain diet. General Mills announced Thursday that it will convert all of its breakfast cereals to whole grain...

Predicted: 0

True: 3

Now that we have gotten positive and negative examples for all 4 classes, we can now display the attention matrices

```
from bertviz import head_view
import matplotlib.pyplot as plt
import seaborn as sns

def plot_cls_attention_heatmap(text, class_id, kind, layer=0, head=0,
                              save=True):
    inputs = tokenizer2.encode_plus(
        text,
        return_tensors='pt',
        add_special_tokens=True,
        truncation=True,
        max_length=128
    )

    input_ids = inputs['input_ids'].to(model2.device)
    attention_mask = inputs['attention_mask'].to(model2.device)

    with torch.no_grad():
        outputs = model2(input_ids=input_ids,
                          attention_mask=attention_mask, output_attentions=True)

    # extract attention for chosen layer and head
    attn = outputs.attentions[layer][0, head].cpu().numpy() # Shape:
```

```

[seq_len, seq_len]
tokens = tokenizer2.convert_ids_to_tokens(input_ids[0])

# extract the CLS row (first row of the attention matrix)
cls_row = attn[0, :] # Only the attention from CLS token to all
tokens

# plot the CLS attention heatmap
plt.figure(figsize=(10, 2))
sns.heatmap(cls_row.reshape(1, -1), cmap="viridis",
xticklabels=tokens, yticklabels=["CLS"], cbar_kws={'label': 'Attention
Weight'}, square=True)
plt.title(f"Class {class_id} - {kind.title()} - Layer {layer},
Head {head}")
plt.xticks(rotation=90)
plt.tight_layout()

if save:
    filename =
f"class{class_id}_{kind}_cls_layer{layer}_head{head}.png"
    plt.savefig(filename, dpi=300)
    print(f"Saved: {filename}")
else:
    plt.show()

plt.close()

# visualization
layer = 4
head = 4

for class_id in range(4):
    pos_text = examples_by_class[class_id]["correct"][0]
    neg_text = examples_by_class[class_id]["incorrect"][0]

    plot_cls_attention_heatmap(pos_text, class_id, "correct",
layer=layer, head=head, save=False)
    plot_cls_attention_heatmap(neg_text, class_id, "incorrect",
layer=layer, head=head, save=False)

```

